FED-NIDS: A Bio-Inspired Federated Deep Learning Framework for Privacy-Preserving Network Intrusion Detection in IoT

Mallem Bourhane Eddine Islam*, Naima Boukhiar†, Pius Sunday Ojwo‡
*Department of Security & Networks – Cybersecurity, EFREI Paris, Paris, France
Email: bourhane-eddine-islam.mallem@efrei.net
†Department of Computer Science, Algiers 1 University, Algiers, Algeria
Email: naima.boukhiar@univ-alger.dz
‡Department of Engineering, University of Constantine 2 – Abdelhamid Mehri, Constantine, Algeria
Email: Sunday.pius@univ-constantine2.dz

***Abstract*—Collaborative intrusion detection for the Internet of Things must accommodate distributed traffic, heterogeneous clients, and restrictions on raw-data sharing. FED-NIDS combines federated orchestration, autoencoder-based traffic representation, and bio-inspired feature and parameter search within a privacy-oriented network intrusion detection framework. On an eight-class, resampled IoT-23-derived dataset, the main 20-client configuration reaches 88.09% centralized-evaluation accuracy and an aggregate F1-score of 0.8784 after seven communication rounds. The final class report contains 1,349,016 observations and reveals substantial class differences: horizontal-port-scan recall is 0.45, compared with 1.00 for DDoS at the report's two-decimal precision. A separate 100-client configuration reaches 83.45% at round 37 and ends at 62.10% after 75 rounds. We distinguish configured from observed participation and server callback timing from training duration. Comparisons with published federated intrusion detection systems identify methodological differences without implying a shared-benchmark ranking. The results demonstrate multiclass detection capability and identify concrete limitations in class coverage and training stability. They do not isolate the gains from individual optimization modules or establish a formal privacy guarantee.**

***Index Terms*—federated learning, network intrusion detection, Internet of Things, bio-inspired optimization, autoencoders, privacy-preserving machine learning**

I. Introduction

IoT network intrusion detection must learn from traffic generated across devices and administrative domains. Pooling training records can be impractical when organizations need to retain control of their data. Federated learning (FL) addresses this constraint by coordinating local model training and exchanging updates rather than requiring a pooled training dataset [1]. It does not eliminate all disclosure risks: gradient information can reveal training examples under particular attack conditions [2]. Detection quality, collaboration, and protection of updates must therefore be considered together.

FED-NIDS brings these concerns into a modular framework developed from our master's project [3]. The framework combines learned traffic representations, federated client/server coordination, bio-inspired feature selection and parameter search, and interfaces for protecting collaborative updates. Our development involved both centrally tracked experiments and practical local iterations. In particular, the optimization work was not completely represented in the tracking service. We distinguish the scope of that development from the narrower question of what each reported numerical experiment measures.

This paper contributes a system-level integration of these elements, an eight-class federated detection evaluation with per-class results, and an analysis of training behavior under two explicitly separated configurations. The seven-round configuration is the main detection result because its final-round report supports class-level analysis. The 75-round configuration provides a separate convergence observation. Their settings and outputs are not combined into a single experiment, and the maximum accuracy along a trajectory is not substituted for its final value.

The evaluation addresses two questions: how effectively the federated detector classifies the eight traffic categories, and how its recorded performance evolves across communication rounds. The framework description provides the context for these measurements. Isolating the effect of each optimization or privacy component is a separate experimental question requiring matched controls.

II. Related Work

A. Federated Intrusion Detection Systems

FedAGRU combines gated recurrent units with attention-based client importance for wireless-edge intrusion detection, with evaluations on WSN-DS, CICIDS2017, and KDD CUP99 [4]. It motivates client-aware collaboration, but its data and protocol differ from our IoT-23-derived classification task.

DeepFed uses convolutional and gated recurrent components for industrial cyber-physical intrusion detection and specifies a Paillier-based protocol protecting model parameters [5]. Its CNN–GRU detector and industrial gas-pipeline evaluation make it a relevant architectural comparison, not a directly interchangeable numerical baseline. DÏoT instead learns device-type-specific behavior profiles without labeled attack exam-

ples [6]. This self-learning anomaly-detection task differs from supervised classification of resampled flows.

SAFE-IDS is a more recent federated IDS that uses noisy gradient-sign aggregation with zSignFedAvg, a unified learning rate, and a weighted loss [7]. Its evaluation spans several IDS datasets. Together, these approaches illustrate different ways to address detection architecture, client heterogeneity, and protection of exchanged information.

B. Positioning and Comparison Boundaries

Table I distinguishes these methodological choices. A reported score on a different label set, distribution, or split is not a controlled comparison with FED-NIDS. Likewise, centralized models are useful context but do not determine whether a gain is attributable to FL, an optimizer, or a privacy mechanism. Our comparison therefore identifies the complementary design choices and explicitly separates literature results from our own observations.

FedProx provides a general FL formulation with proximal regularization for heterogeneous clients [8]. We use it as a framework-level optimization option rather than presenting it as a new contribution. The contribution of the bio-inspired layer is its organization around the detector's feature, parameter, and architecture choices. Its individual performance gains require matched component-on/off experiments; the present results do not constitute those ablations.

III. FED-NIDS Framework

A. Architecture and Collaborative Objective

Figure 1 shows the framework's local detection, optimization, federation, and deployment paths. Each client maintains flow-derived features and a local detector. The server coordinates model distribution and aggregation. With client data D_k, sample count n_k, and loss L_k, the standard federated objective is

$$\min_{\theta} F(\theta) = \sum_{k=1}^{K} \frac{n_k}{\sum_j n_j} L_k(\theta). \tag{1}$$

The sample-weighted objective defines collaboration over the client populations [1]; it does not establish that every configured client returns a result in every round. The actual response counts are reported separately in Section IV.

The orchestration uses Flower with PyTorch models and simulated clients [3], [9]. The FedProx option modifies a client's local objective to

$$L_k(\theta) + \frac{\mu}{2}\|\theta - \theta^{(t)}\|_2^2, \tag{2}$$

where $\theta^{(t)}$ is the broadcast model and μ penalizes departure from it [8]. This is a regularizer, not a hard bound on parameter movement. The framework also considers attentive client weighting. The reported run configurations do not identify the full client-loss or weighting implementation, so we do not assign an unrecorded coefficient or a component-specific effect to their outcomes.

B. Traffic Representation and Classification

The detector couples an autoencoder with a feedforward classifier. The stored configurations identify 15 input features, a 32-dimensional encoded representation, a hidden-size parameter of 64, and eight output classes [10]. An encoder transforms the feature vector into a learned representation and a decoder supports reconstruction learning; the classifier produces traffic-category predictions. The development report describes autoencoder pretraining followed by federated learning [3].

Because 32 exceeds 15, this representation is not a dimensionality reduction. Nor does that dimension alone establish a smaller communication payload. Communication cost depends on the tensors actually exchanged and their encoding. Reconstruction error is a useful design signal, but the reported eight-class results concern the classifier; they do not independently evaluate a reconstruction-threshold anomaly detector.

C. Bio-Inspired Development and Optimization Roles

Bio-inspired optimization was part of our practical development, including iterations performed outside centrally logged training. The development report describes bio-inspired feature selection after preliminary correlation-based filtering [3, Sec. 6.3.2]. The framework organizes the search into four complementary strategies, summarized in Table II. These roles follow the system design; they should not be read as four separately evaluated treatments in the runs reported here.

Feature selection, learning-rate selection, and architecture selection affect different objects in the training pipeline. We therefore describe them at the module level rather than attributing a final accuracy to an unspecified mixture of searches. For example, PSO-based refinement uses particle positions for candidate settings and attraction toward locally and globally favorable candidates; genetic variation provides an alternative source of candidate diversity. In FED-NIDS these mechanisms concern configuration search, not a replacement definition of the federated aggregation objective [3].

The numerical study evaluates the detector in two fixed recorded configurations. It does not isolate the marginal effect of ASSCO, ISSA–HHO, PSO–GA, or GWO–ABC, and it does not claim that all four were active within each logged round. This scope is compatible with practical development outside the tracking service. It separates the existence of development work from a causal estimate of its contribution to the reported metrics.

D. Privacy Objectives and Deployment Scope

The framework's privacy objective is to retain client training records locally while coordinating learning through model updates. Formal privacy and secure communication address additional risks. Differential privacy limits the change in an output distribution under a defined change to the protected dataset and requires a specified mechanism, sensitivity control, and accounting across releases [11]. Secure aggregation instead protects individual contributions while revealing their

TABLE I
COMPARISON WITH PUBLISHED FL-IDS DESIGNS. ENTRIES SUMMARIZE THE ORIGINAL METHODS, NOT EXPERIMENTS REPRODUCED UNDER A COMMON FED-NIDS PROTOCOL.

System	Detector/collaboration	Evaluation setting	Relationship to FED-NIDS
FedAGRU [4]	GRU-based detection; attention-based client importance.	WSN-DS, CICIDS2017, KDD CUP99.	Relevant client-weighting design; different data and preprocessing.
DeepFed [5]	CNN–GRU; Paillier-protected parameter exchange.	Industrial gas-pipeline intrusion detection.	Explicit protection protocol; different industrial task.
DÏoT [6]	Federated device-type behavior profiles.	IoT-device anomaly detection without labeled attacks.	Different supervision and unit of evaluation.
SAFE-IDS [7]	DNN; noisy sign aggregation and weighted loss.	CIC-IDS2017/2018, CIC-DDoS2019, KDDCup99, NSL-KDD.	Relevant heterogeneity/privacy design; different evaluation partitions.
FED-NIDS	Autoencoder–classifier; modular bio-inspired optimization and privacy interfaces.	Eight-class resampled IoT-23 derivative; separate 20-/100-client runs.	Run-linked accuracy and class report; component ablations and matched FL-IDS controls remain outside this evaluation.

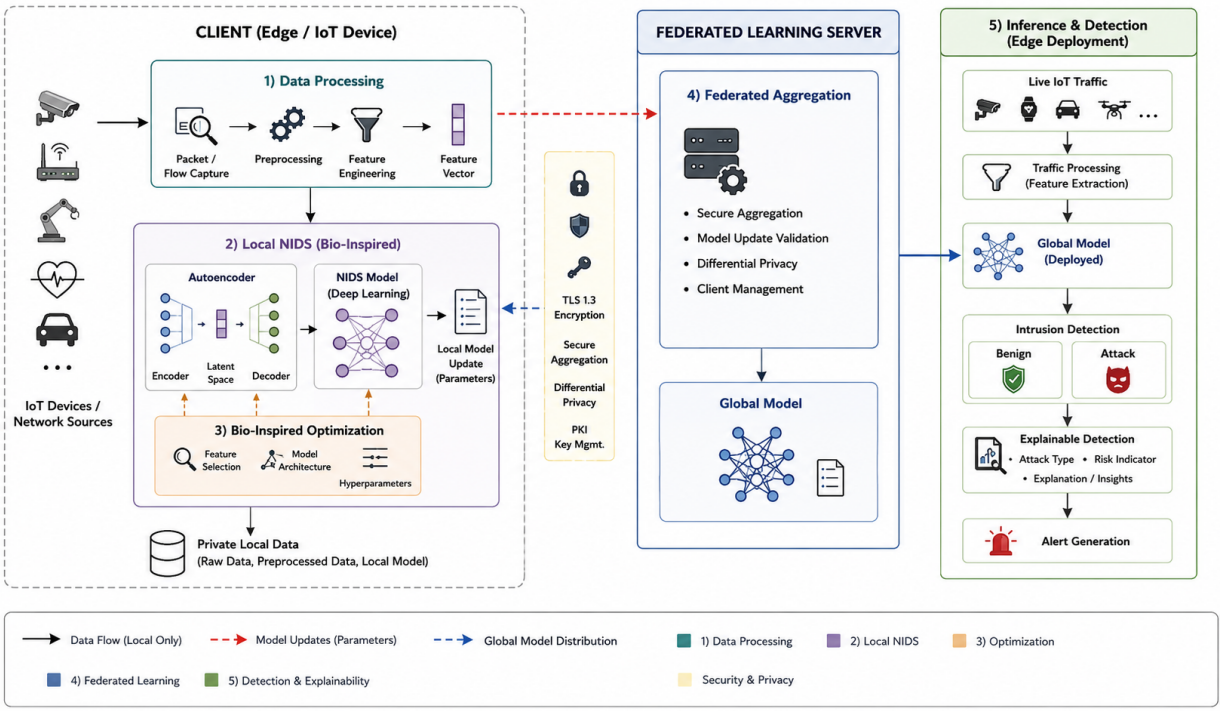


Fig. 1. FED-NIDS architecture: local flow processing and detection, bio-inspired optimization, federated coordination, and deployment/security interfaces. The figure describes the system organization; it is not a per-run activation diagram or a claim that every displayed deployment capability was separately benchmarked.

aggregate under a stated protocol and threat model [12]. These protections are distinct from encryption in transit.

FED-NIDS includes these concerns in its privacy-oriented architecture. The quantitative evaluation does not report a composed privacy budget, a measured privacy attack, or a run-linked cryptographic benchmark. We therefore do not interpret detection accuracy as a demonstrated privacy guarantee or a measured privacy–utility trade-off. Similarly, the deployment path in Fig. 1 describes intended operation at the edge; the present experiments do not measure live-traffic latency, explainability quality, energy use, or resistance to poisoned clients.

IV. EXPERIMENTAL SETUP AND REPORTING PROTOCOL

A. Dataset and Evaluation Population

IoT-23 provides labeled benign and malicious traffic captured from IoT environments [13]. Our configurations identify an *Aposemat IoT-23 Kaggle Dataset (Resampled)* derivative rather than an unmodified benchmark release [10]. The development workflow includes cleaning, correlation-

TABLE II
BIO-INSPIRED SEARCH ROLES IN THE FED-NIDS FRAMEWORK [3, SEC. 5.3.6]. NO PER-STRATEGY ACCURACY INCREMENT IS ESTIMATED.

Strategy	Role in development and system design
ASSCO	Feature-subset and parameter search using social-spider-inspired candidate interaction.
ISSA–HHO	Exploration/exploitation balancing for detector parameter search.
PSO–GA	Parameter search combining particle-based refinement and genetic variation.
GWO–ABC	Hierarchical candidate search and exploration of promising architecture/parameter configurations.

TABLE III
SEPARATE RUN CONFIGURATIONS AND FINAL-ROUND OBSERVATIONS. CONFIGURED PARTICIPATION FIELDS ARE NOT ASSUMED TO EQUAL RECEIVED RESULTS.

Item	A	B
Client population	20	100
Communication-round budget	7	75
Local epochs	10	3
Learning rate	0.001	0.01
Pretraining epochs	10	1
Stored fit-participation field	10	15
Stored evaluation-participation field	5	50
Received fit results, final round	20	15
Distributed evaluation, final round	20 results	Skipped

based filtering, categorical encoding, scaling, feature selection, and SMOTE-based balancing [3]. Its nominal training/validation/test division is 60/20/20; exact run-linked split indices and preprocessing versions are not retained with the records considered here.

The eight classes are Attack, Benign, C&C, C&C-HeartBeat, DDoS, HeartBeat, Okiru, and PartOfAHorizontalPortScan. The main final-round report has 168,627 observations in each class, totaling 1,349,016. Consequently, these results characterize a balanced evaluation population, not natural attack prevalence. We do not infer an unseen-class mapping from other development figures or claim capture-disjoint evaluation.

The seven-round output is identified as Flower's *centralized evaluation*; its log does not explicitly name the split as test. The 75-round log explicitly prints *Test Accuracy*. This terminology is retained. Centralized evaluation of a federated model should not be confused with centralized model training or with an average of distributed client-evaluation metrics.

B. Hardware and Run Configuration

Both records identify Python 3.12.7, macOS 14.6.1, and an Apple M1 Pro with 16 GB memory. The stored tracking configuration names the MPS device, while the application device field is `auto`; this is configuration metadata rather than a device utilization trace. Batch size 64 and the model dimensions described above are common to both runs. Table III lists their differing settings.

Record A is `gsunfh9x` (`pious-elevator-5`); record B is `xjbhw3rt` (`peach-wave-26`) [10]. In A, round 7 receives 20 fit results with zero failures and 20 distributed-evaluation results. Round 1 has 17 fit results and three failures. Therefore, the stored 10/5 participation fields cannot be described as the actual number participating in every round. In B, the quoted rounds 1 and 75 each receive 15 fit results; at round 75 distributed evaluation is explicitly skipped. Its centrally evaluated accuracy is not an average over the configured 50 evaluation clients.

The stored fit configuration includes momentum 0.9 but no explicit optimizer name for these two runs. Although the development report discusses Adam, this field alone does not establish optimizer identity. We report the recorded learning rate and local epochs without inferring Adam-specific settings or importing parameters from later runs. The two configurations differ in several factors, so they do not isolate a client-count or scalability effect.

C. Metrics and Round Selection

Accuracy is the proportion of correctly classified observations. For class c, precision and recall are $P_c = TP_c/(TP_c + FP_c)$ and $R_c = TP_c/(TP_c + FN_c)$; F1 is $2P_cR_c/(P_c + R_c)$. We retain the class report's original two-decimal precision and the separately logged aggregate scalars. Equal class supports make macro and support-weighted class averages coincide, but the high-precision scalar dictionary does not explicitly name its averaging option.

Round 0 denotes initialization evaluation. Final accuracy is the value at the last completed configured round, while peak accuracy is the maximum observed in that run's trajectory. The peak is a retrospective descriptor, not evidence of validation-based checkpoint selection. A logged F-beta value is not used to claim a model-selection rule because its beta and validation role are not specified. Replayed metric rows are counted once, using the explicit communication-round identifier.

V. RESULTS AND DISCUSSION

A. Main Seven-Round Detection Result

Record A increases from 12.50% at initialization to 60.46% after round 1 and 83.46% after round 2. Its endpoint is 88.09% at round 7 (Fig. 2). The recorded aggregate precision, recall, and F1 at that round are 0.9086, 0.8809, and 0.8784, respectively. The F1 value is reported as a logged aggregate, not newly recomputed from rounded class entries. Most of the observed increase occurs early, with only about 0.04 percentage points between rounds 5 and 7; this is a description of one trajectory, not a statistical convergence guarantee.

Table IV reveals a more informative picture than accuracy alone. Horizontal-port-scan recall is 0.45 with F1 0.61, identifying limited coverage of that class in this evaluation. Benign recall is 0.95, but benign precision is 0.59: many observations assigned the benign label belong to another class. Without the confusion-matrix counts, we cannot identify exactly which classes account for those assignments. DDoS and Okiru have

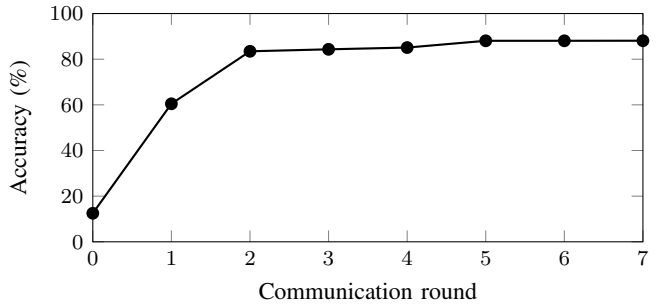


Fig. 2. Record A's complete centralized-evaluation accuracy trajectory. These are eight round-indexed observations from one training run, not independent repetitions.

TABLE IV
RECORD A, ROUND 7: CLASS-LEVEL RESULTS AT THEIR ORIGINAL TWO-DECIMAL PRECISION. EVERY CLASS HAS SUPPORT 168,627; TOTAL SUPPORT IS 1,349,016.

Class	Precision	Recall	F1
Attack	1.00	0.93	0.96
Benign	0.59	0.95	0.72
C&C	0.91	0.80	0.85
C&C-HeartBeat	1.00	0.92	0.96
DDoS	1.00	1.00	1.00
HeartBeat	0.87	1.00	0.93
Okiru	1.00	1.00	1.00
PartOfAHorizontal-PortScan	0.92	0.45	0.61
Macro average (printed)	0.91	0.88	0.88
Weighted average (printed)	0.91	0.88	0.88

precision, recall, and F1 printed as 1.00; the rounded report does not establish zero errors.

These differences motivate class-specific reporting for a multiclass NIDS. The system recognizes several traffic categories well under the balanced evaluation, while weaker scan recall and benign precision remain concrete priorities. Equal evaluation supports ensure that the weak classes are visible rather than numerically overwhelmed by a majority class; they do not establish performance under an operational traffic distribution.

B. Separate 75-Round Configuration

Table V reports selected observations from B's complete 76-entry trajectory, including its maximum and endpoint. The maximum is 83.45% at round 37; final accuracy is 62.10% at round 75. The peak-to-final difference is 21.36 percentage points.

The decline shows that a larger round budget did not guarantee retention of the best observed accuracy in this configuration. It does not identify whether the cause was a learning-rate choice, partition structure, optimizer behavior, or another implementation detail. Because A and B use different local-training and pretraining budgets, their endpoints are not a controlled comparison of client population or communication efficiency.

TABLE V
RECORD B: SELECTED CENTRAL TEST-EVALUATION OBSERVATIONS. ROUND 37 IS THE MAXIMUM, NOT A SELECTED VALIDATION CHECKPOINT.

Round	Accuracy (%)	Logged loss	Position
0	14.57	2.0766	Initialization
1	68.14	0.9804	Intermediate
25	81.88	0.5211	Intermediate
37	83.45	0.5907	Maximum accuracy
50	80.07	0.8439	Intermediate
75	62.10	0.8804	Final round

TABLE VI
CENTRALIZED NEURAL REFERENCE RESULTS FROM THE DEVELOPMENT STUDY [3, TABLE 16]: 10 EPOCHS AND BATCH SIZE 64. THE SOURCE TABLE DOES NOT SPECIFY THE PRECISION/RECALL/F1 AVERAGING CONVENTION.

Model	Acc. (%)	Prec.	Recall	F1
FFN + autoencoder	84.9	0.869	0.838	0.841
GRU + autoencoder	85.0	0.887	0.854	0.841
LSTM + autoencoder	85.1	0.898	0.857	0.847
CNN + autoencoder	85.9	0.873	0.832	0.825

C. Training Duration and Callback Measurements

A's Flower summary reports seven rounds completed in 8,448.50 s. Its separately logged training-time quantity is 8,900.49 s, and the run-summary runtime is 8,912.77 s. These quantities have different instrumentation scopes and are not interchangeable. In particular, A's round-indexed accuracy rows were written together near run completion; their adjacent timestamps are not per-round execution durations.

For B, the stored quantity named `Total Training Time` is 0.912394 s. The archive inventory identifies it as the sum of 75 short callback timings, giving a mean of about 0.012165 s per callback [10]. The console places these short measurements after fit aggregation and before central evaluation. Exact timer boundaries would require the original instrumentation code. In contrast, the stored final run-summary runtime is 15,466.86 s. We therefore report the small quantity as a callback subtotal, not elapsed training or 0.912 s per round. No end-to-end speedup over centralized learning is inferred.

D. Centralized Reference Results

The development study also evaluated centralized classifiers [3]. Gradient boosting achieved 98.6% accuracy, compared with 97.7% for random forest, 97.2% for a decision tree, and 86.7% for K-nearest neighbors. Table VI retains the autoencoder-paired neural results, including the recall column. These provide context for the detector family, but their exact partition identities are not linked to A and B. Consequently, differences between the centralized and federated numbers do not isolate a cost or benefit of federation, privacy, or bio-inspired search.

VI. Limitations and Reproducibility

The results concern historical simulated clients and a single IoT-23-derived dataset. The two identified configurations are not independent-seed repetitions, and seed-controlled variability is not available. Successive rounds and duplicate logged histories are not substitutes for repetitions. The evaluation therefore reports individual runs without mean–standard-deviation estimates or significance claims.

Per-class recall and F1 strengthen the main evaluation, but prediction scores, class-linked probability arrays, and numerical confusion-matrix counts were not retained in the available records. Multiclass AUROC needs score-level information [14] and is not reconstructed from accuracy or a rounded classification report. Exact preprocessing/split manifests are also needed to determine how the balanced evaluation was constructed; neither absence nor presence of leakage is established by the available record alone.

The framework's practical optimization work is broader than these two tracked runs. Their metrics do not isolate each metaheuristic's contribution, quantify privacy guarantees, or demonstrate deployment performance. Future evaluation should pair documented component activation with matched FL baselines, training-only transformation fitting, explicit class and split mappings, validation-based selection, and independently seeded repetitions. These are extensions to the reported study, not additional experiments claimed here.

The accompanying author materials retain the run identifiers, extracted configuration and metric tables, original reporting precision, and deterministic calculations. A's source commit was not stored; B records commit `4a14f782` of the experiment repository. These identifiers support traceability without implying that every development iteration was centrally logged. No model retraining was performed to obtain the corrections or class report presented here.

VII. Conclusion

FED-NIDS brings federated model coordination, learned traffic representations, and practical bio-inspired development into a privacy-oriented IoT intrusion detection framework. Its main seven-round evaluation reaches 88.09% accuracy and aggregate F1 0.8784 over a balanced eight-class population. Per-class reporting identifies horizontal-port-scan recall and benign precision as important limitations despite strong results for several other categories. A separate 75-round configuration reaches 83.45% at round 37 but ends at 62.10%, showing why endpoint and maximum-observed performance must be distinguished. By linking results to their configurations and separating callback timing from training duration, the evaluation offers a clearer basis for subsequent component ablation, matched comparisons, and deployment validation.

References

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, PMLR, vol. 54, pp. 1273–1282, 2017.

[2] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[3] Pius Sunday Ojwo and Mallem Bourhane Eddine Islam, "FED-NIDS: A bio-inspired federated deep learning-based network intrusion detection system for IoT ecosystem security," Master's project report, University of Constantine 2–Abdelhamid Mehri, 2024, chs. 5–7.

[4] Z. Chen, N. Lv, P. Liu, Y. Fang, K. Chen, and W. Pan, "Intrusion detection for wireless edge networks based on federated learning," *IEEE Access*, vol. 8, pp. 217463–217472, 2020, doi: 10.1109/ACCESS.2020.3041793.

[5] B. Li, Y. Wu, J. Song, R. Lu, T. Li, and L. Zhao, "DeepFed: Federated deep learning for intrusion detection in industrial cyber-physical systems," *IEEE Trans. Industrial Informatics*, vol. 17, no. 8, pp. 5615–5624, 2021, doi: 10.1109/TII.2020.3023430.

[6] T. D. Nguyen, S. Marchal, M. Miettinen, H. Fereidooni, N. Asokan, and A.-R. Sadeghi, "DÏoT: A federated self-learning anomaly detection system for IoT," in *Proc. IEEE ICDCS*, 2019, pp. 756–767, doi: 10.1109/ICDCS.2019.00080.

[7] Alimov Abdulboriy Abdulkhay ugli and Ji Sun Shin, "SAFE-IDS: A privacy-preserving framework for overcoming non-IID challenges in federated intrusion detection," *Computers & Security*, vol. 155, Art. no. 104492, 2025, doi: 10.1016/j.cose.2025.104492.

[8] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Machine Learning and Systems*, vol. 2, 2020.

[9] D. J. Beutel et al., "Flower: A friendly federated learning research framework," arXiv:2007.14390, 2020.

[10] FED-NIDS project, "Federated experiment records: `gsunfh9x` and `xjbhw3rt`," Weights & Biases project `piussunday-dev/FED-NIDS Federated`, Dec. 2024. Stored configurations, round histories, and Flower console excerpts; project records, not a separately published dataset.

[11] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Foundations and Trends in Theoretical Computer Science*, vol. 9, nos. 3–4, pp. 211–407, 2014, doi: 10.1561/0400000042.

[12] K. Bonawitz et al., "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM CCS*, 2017, pp. 1175–1191, doi: 10.1145/3133956.3133982.

[13] S. Garcia, A. Parmisano, and M. J. Erquiaga, "IoT-23: A labeled dataset with malicious and benign IoT network traffic," ver. 1.0.0, Zenodo, 2020, doi: 10.5281/zenodo.4743746.

[14] Scikit-learn developers, "`roc_auc_score`," API documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_auc_score.html. Accessed: Sep. 17, 2026.